Overview

The assessment involves object detection in a retail setting using image detection models. This can prove to be quite a useful approach to taking stock counts in a retail store at the end of quarters, which typically requires time-consuming manual counting.

The task requires building a model which delivers acceptable metrics in detecting all the required stocks while also making sure the items detected and classified are as accurate as possible. This would in turn make the model suitable in the practical world.

The task also requires displaying product dominance in a collective shelf.

The Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import yaml

yaml_path = "/content/drive/MyDrive/dataset/data.yaml"

with open(yaml_path, "r") as f:
    data = yaml.safe_load(f)

!cat {yaml_path}
```

```
train: ../train/images
val: ../valid/images
test: ../test/images

nc: 76
names: ['q1', 'q10', 'q100', 'q103', 'q106', 'q109', 'q112', 'q115', 'q118',

roboflow:
  workspace: project-6jait
  project: final_try
  version: 4
  license: CC BY 4.0
  url: https://universe.roboflow.com/project-6jait/final_try/dataset/4
```

The dataset is being loaded from google drive, as the assignment is being done from Google Colab. This requires tweaking of the data.yaml file to ensure proper access.

```
new_data = {
    "path": "/content/drive/MyDrive/dataset",
    "train": "train/images",
    "val": "valid/images",
    "test": "test/images",
    "nc": data["nc"],
    "names": data["names"],
    "roboflow": data["roboflow"]
}
```

There is a total of 76 classes of stock, with a unique code for each class

```
with open(yaml_path, "w") as f:
    yaml.dump(new_data, f, sort_keys=False, default_flow_style=False)
```

Exploratory Data Analysis

Let us look at the training data

```
import os

dataset_path = "/content/drive/MyDrive/dataset/train"
image_folder = os.path.join(dataset_path, "images")
label_folder = os.path.join(dataset_path, "labels")
```

```
files = sorted(os.listdir(image_folder))
print("Training images: ", files[:10])


files = sorted(os.listdir(label_folder))
print("Training labels: ", files[:10])
```

```
Training images:  ['db1021_jpg.rf.58ac3404fb4f7c4c9e76a628ea49b1ac.jpg', 'db1
Training labels:  ['db1021_jpg.rf.58ac3404fb4f7c4c9e76a628ea49b1ac.txt', 'db1
```

The training data consists of images and labels, possibly pointing to the images and their annotations in the YOLO format. Let us explore some random training examples.

```
import cv2
import random
import matplotlib.pyplot as plt
```

```
import os


with open(yaml_path, "r") as f:
    data = yaml.safe_load(f)

class_names = data["names"]


image_files = os.listdir(image_folder)
random_image = random.choice(image_files)

img_path = os.path.join(image_folder, random_image)
label_path = os.path.join(label_folder, random_image.replace(".jpg", ".txt

img = cv2.imread(img_path)
h, w, _ = img.shape

# Draw bounding boxes
if os.path.exists(label_path):
    with open(label_path, "r") as f:
        for line in f.readlines():
            cls, x, y, bw, bh = map(float, line.split())

            # Convert YOLO format to pixel coordinates
            x1 = int((x - bw/2) * w)
            y1 = int((y - bh/2) * h)
            x2 = int((x + bw/2) * w)
            y2 = int((y + bh/2) * h)

            class_id = int(cls)
            label = class_names[class_id]

            cv2.rectangle(img, (x1, y1), (x2, y2), (0,255,0), 2)

            # Draw label background
            cv2.rectangle(img, (x1, y1 - 20), (x2, y1), (0,255,0), -1)

            # Put class text
            cv2.putText(
                img,
                label,
                (x1 + 5, y1 - 5),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                (0,0,0),
                1,
                cv2.LINE_AA
            )

plt.figure(figsize=(8,8))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

The images look uncorrupted, while the annotations are more or less close to the target image.

Let us check for missing data in the dataset

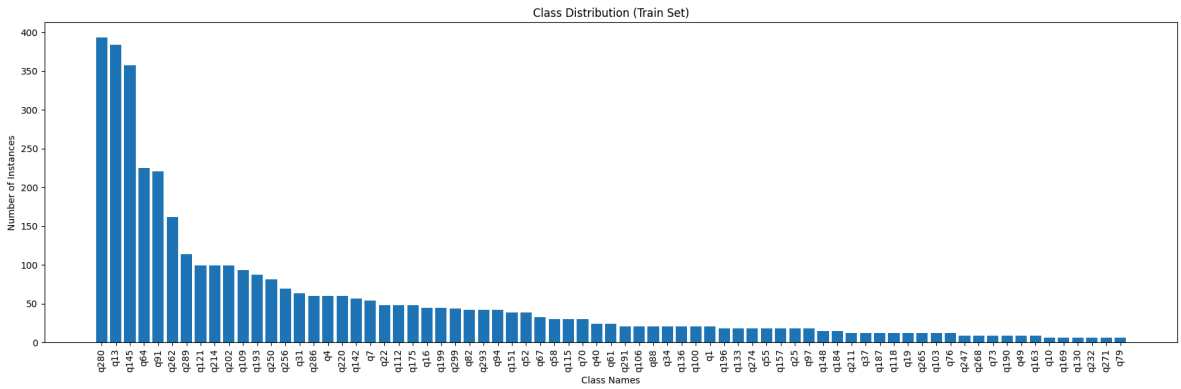
```
total_images = 0
total_labels = 0

for split in ['train', 'valid', 'test']:
    # img_count = len(os.listdir(f'{split}/images')) # use this line if lc
    # label_count = len(os.listdir(f'{split}/labels')) # use this line if
    img_count = len(os.listdir(f'/content/drive/MyDrive/dataset/{split}/im
    label_count = len(os.listdir(f'/content/drive/MyDrive/dataset/{split}/

    print(f"{split.upper()} -> Images: {img_count}, Labels: {label_count}"

    total_images += img_count
    total_labels += label_count

print("\nTotal Images:", total_images) # printing total image count
print("Total Labels:", total_labels) # printing total label count
```

```
TRAIN -> Images: 924, Labels: 924
VALID -> Images: 40, Labels: 40
TEST -> Images: 35, Labels: 35

Total Images: 999
Total Labels: 999
```

There is no missing data, and the train data consists of ~92.5% of the dataset, while valid and test sets have ~4% and ~3.5% respectively.

Let us check for class imbalance

```
from collections import Counter

# Count class occurences
class_counts = Counter()

for file in os.listdir(label_folder):
    with open(os.path.join(label_folder, file), "r") as f:
        for line in f:
            cls_id = int(line.split()[0])
            class_name = class_names[cls_id]
            class_counts[class_name] += 1

# Sort for cleaner plotting
class_counts = dict(sorted(class_counts.items(), key=lambda x: x[1], rever

# Plot
plt.figure(figsize=(18,6))
plt.bar(class_counts.keys(), class_counts.values())

plt.xticks(rotation=90)
plt.xlabel("Class Names")
plt.ylabel("Number of Instances")
plt.title("Class Distribution (Train Set)")
plt.tight_layout()
plt.show()
```



There seems to be a noticeable class imbalance in the training dataset, which could affect generalisation capability of the trained model when subjected to new instances

of classes with low occurences in the training set. We will try to see if we can produce an acceptable trained model with the data we currently have, and look towards further methods if not.

Problem Statement

The current recall is 67.6%, meaning the model misses about 32% of the items on the shelf. In a real store, this would lead to incorrect stock counts.

Model Optimization: Increase the Recall metric without significantly damaging the Precision.

Approach

We need to train a model with a resulting recall metric higher than 67.6%. Recall refers to a model's ability to detect instances of each class. A model with a high recall value would therefore detect most instances of each class (true positives) and miss as little as possible (false negatives).

A low recall is the result of a number of reasons:

- Small products in images being missed
- Overlapping SKU's being suppressed
- Low-confidence detections being discarded
- Class imbalance

We can adopt a number of methods to try and increase recall

- **Lower confidence threshold**: Setting the percentage at which a model is confident about the presence of an object to a low value would mean the model would miss less products during detection. This is important in a retail setting as items are usually hidden or are covered by other items in shelves.
- **Increase Image Resolution**: Increasing image resolution would help detect small items, often the case in retail stores.
- **Train Longer (with Early Stopping)**: Training a model longer might help when the dataset is large. However we need to keep care that the model does not overfit by implementing early stopping conditions.
- **Handling Class Imbalance**: Using loss functions which target class imbalance could increase recall, especially in our case where some SKUs appear relatively rarely in the training images.

Our Approach: We will train a baseline model using default out-of-the package settings, and then train a model with higher resolution and longer epochs. While

evaluating we will try out different confidence thresholds on the test data to see which threshold yields higher recall with less damaging precision.

Model training

The dataset is a good example of a YOLO(You only look once) dataset, with text files containing annotations of the each image. In this case, a YOLO image processing model would be an obvious choice, due to its compatibility with such datasets, and its capability to process images in real time being an excellent advantage in retail environments, where keeping real-time counts of product is a necessity.

We will look towards the YOLO models from the [ultralytics library](). The library comes with user-friendly detailed instructions for diverse use-cases.

```
!pip install ultralytics #installing ultralytics library
```

```
from ultralytics import YOLO

model_base = YOLO("yolov8m.pt")
model_base.train(
    data = yaml_path,
    epochs = 100,
    imgsz = 640,
    name = 'Baseline training',
    seed = 7
)
```

```
0.34134,     0.34234,     0.34334,     0.34434,     0.34535,     0.34635,
0.34735,     0.34835,     0.34935,     0.35035,     0.35135,     0.35235,
0.35335,     0.35435,     0.35536,     0.35636,     0.35736,     0.35836,
0.35936,
          0.36036,     0.36136,     0.36236,     0.36336,     0.36436,
0.36537,     0.36637,     0.36737,     0.36837,     0.36937,     0.37037,
0.37137,     0.37237,     0.37337,     0.37437,     0.37538,     0.37638,
0.37738,     0.37838,     0.37938,     0.38038,     0.38138,     0.38238,
0.38338,
          0.38438,     0.38539,     0.38639,     0.38739,     0.38839,
0.38939,     0.39039,     0.39139,     0.39239,     0.39339,     0.39439,
0.3954,      0.3964,      0.3974,      0.3984,      0.3994,      0.4004,
0.4014,      0.4024,      0.4034,      0.4044,     0.40541,     0.40641,
0.40741,
          0.40841,     0.40941,     0.41041,     0.41141,     0.41241,
0.41341,     0.41441,     0.41542,     0.41642,     0.41742,     0.41842,
0.41942,     0.42042,     0.42142,     0.42242,     0.42342,     0.42442,
0.42543,     0.42643,     0.42743,     0.42843,     0.42943,     0.43043,
0.43143,
          0.43243,     0.43343,     0.43443,     0.43544,     0.43644,
0.43744,     0.43844,     0.43944,     0.44044,     0.44144,     0.44244,
0.44344,     0.44444,     0.44545,     0.44645,     0.44745,     0.44845,
0.44945,     0.45045,     0.45145,     0.45245,     0.45345,     0.45445,
0.45546,
          0.45646,     0.45746,     0.45846,     0.45946,     0.46046,
0.46146,     0.46246,     0.46346,     0.46446,     0.46547,     0.46647,
0.46747,     0.46847,     0.46947,     0.47047,     0.47147,     0.47247,
0.47347,     0.47447,     0.47548,     0.47648,     0.47748,     0.47848,
0.47948,
          0.48048,     0.48148,     0.48248,     0.48348,     0.48448,
0.48549,     0.48649,     0.48749,     0.48849,     0.48949,     0.49049,
0.49149,     0.49249,     0.49349,     0.49449,      0.4955,      0.4965,
0.4975,      0.4985,      0.4995,      0.5005,      0.5015,      0.5025,
```

```
!cp -r '/content/runs/detect/Baseline training' '/content/drive/MyDrive/Reta
```

```
model_high_res = YOLO("yolov8m.pt")
model_high_res.train(
    data = yaml_path,
    epochs = 150,
    imgsz = 1024,
    name = 'High_res training',
    seed = 7,
    patience = 50
)
```

```
maps: array([    0.51625,     0.56879,       0.796,     0.51625,
0.8955,     0.43456,     0.32407,     0.55019,      0.8233,     0.51625,
0.44425,     0.51625,     0.51625,     0.51625,     0.51625,     0.29937,
0.51625,     0.20951,     0.67526,           0,     0.51625,     0.51625,
0.18475,           0,
          0.51625,      0.6965,     0.51625,     0.51625,     0.51625,
0.51625,     0.51625,     0.51625,       0.796,     0.51625,     0.51625,
0.67605,           0,     0.51625,      0.8955,     0.51625,     0.56427,
0.2797,     0.59149,       0.796,     0.51625,     0.51625,     0.51625,
0.64703,
          0.48112,     0.51625,     0.51625,     0.51625,     0.51625,
0.49377,     0.51625,     0.51625,     0.52601,     0.51625,           0,
0.62881,      0.6965,     0.51625,     0.51625,     0.79602,     0.51625,
0.74964,     0.51625,     0.51625,     0.51625,     0.51625,     0.51625,
0.51625,
          0.51625,     0.51625,     0.51625,     0.51625])
names: {0: 'q1', 1: 'q10', 2: 'q100', 3: 'q103', 4: 'q106', 5: 'q109', 6:
'q112', 7: 'q115', 8: 'q118', 9: 'q121', 10: 'q13', 11: 'q130', 12: 'q133',
13: 'q136', 14: 'q142', 15: 'q145', 16: 'q148', 17: 'q151', 18: 'q157', 19:
'q16', 20: 'q163', 21: 'q169', 22: 'q175', 23: 'q178', 24: 'q184', 25:
'q187', 26: 'q19', 27: 'q190', 28: 'q193', 29: 'q196', 30: 'q199', 31:
'q202', 32: 'q211', 33: 'q214', 34: 'q22', 35: 'q220', 36: 'q229', 37:
'q232', 38: 'q247', 39: 'q25', 40: 'q250', 41: 'q256', 42: 'q262', 43:
'q265', 44: 'q268', 45: 'q271', 46: 'q274', 47: 'q280', 48: 'q286', 49:
'q289', 50: 'q291', 51: 'q293', 52: 'q299', 53: 'q31', 54: 'q34', 55:
'q37', 56: 'q4', 57: 'q40', 58: 'q46', 59: 'q49', 60: 'q52', 61: 'q55', 62:
'q58', 63: 'q61', 64: 'q64', 65: 'q67', 66: 'q7', 67: 'q70', 68: 'q73', 69:
'q76', 70: 'q79', 71: 'q82', 72: 'q88', 73: 'q91', 74: 'q94', 75: 'q97'}
nt_per_class: array([ 0,  4,  1,  0,  3,  3,  3,  4,  2,  0, 44,  0,  0,
0,  0, 12,  0,  2,  6,  3,  0,  0,  9,  2,  0,  1,  0,  0,  0,  0,  0,  0,
1,  0,  0,  3,  4,  0,  1,  0,  3,  5,  4,  2,  0,  0,  0, 32,  4,  0,  0,
0,  0, 22,  0,  0,  4,  0,  5,  5,  1,  0,  0,  2,  0,  2,  0,  0,  0,  0,
0,  0,  0,  0,  0,  0])
nt_per_image: array([0, 2, 1, 0, 1, 1, 2, 3, 2, 0, 4, 0, 0, 0, 0, 2, 0, 1,
1, 1, 0, 0, 2, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 2, 0, 1, 0, 1, 2, 1,
1, 0, 0, 0, 5, 2, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 2, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0])
results_dict: {'metrics/precision(B)': 0.7471129357317579,
'metrics/recall(B)': 0.6767345688760019, 'metrics/mAP50(B)':
0.7379315484111406, 'metrics/mAP50-95(B)': 0.5162485198517115, 'fitness':
0.5162485198517115}
save_dir: PosixPath('/content/runs/detect/High_res training')
speed: {'preprocess': 0.32254402499347634, 'inference': 3.8488239499884003,
'loss': 0.00039482499687437667, 'postprocess': 0.8853238249912465}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [],
'target_img': []}
task: 'detect'
```

```
!cp -r '/content/runs/detect/High_res training' '/content/drive/MyDrive/Reta
```

Model training with the high-resolution images stopped early at 67 epochs due to no improvements for more than 50 epochs.

Evaluation

We will evaluate the two trained models using varying confidence thresholds. Threshold will start with 0.25 and decrease till a high value of recall is achieved without damaging the precision value significantly.

Moreover, we will be looking at the mAP50-95 (mean Average Precision across multiple IoU thresholds from 0.5 to 0.95)

```
from ultralytics import YOLO

model_base = YOLO("/content/drive/MyDrive/Retail Assessment/Baseline trainin
model_high_res = YOLO("/content/drive/MyDrive/Retail Assessment/High_res tra
```

```
metrics_base_25 = model_base.val(
    data= yaml_path,
    split="test",
    conf=0.25,
    iou=0.6
)
```

```
Ultralytics 8.4.19 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100-S
Model summary (fused): 93 layers, 25,883,764 parameters, 0 gradients, 78.9 GF
val: Fast image access ✅ (ping: 0.4±0.1 ms, read: 0.4±0.1 MB/s, size: 101.1
val: Scanning /content/drive/MyDrive/dataset/test/labels.cache... 35 images,
                 Class     Images  Instances      Box(P          R      mAP50
                   all         35        145      0.757      0.777      0.811
                  q106          1          3          1          1      0.995
                  q109          1          3          1          1      0.995
                  q112          2          3          1      0.333      0.667
                  q115          1          1          1          1      0.995
                  q121          1          4      0.444          1      0.912
                   q13          1          8      0.727          1      0.995
                  q142          1          3          1          1      0.995
                   q16          1          1          0          0          0
                  q163          1          1          1          1      0.995
                  q184          1          1          1          1      0.995
                  q187          2          3          1      0.333      0.667
                  q193          1         10      0.529        0.9      0.894
                  q199          1          3          1          1      0.995
                  q202          1          3       0.75          1      0.995
                  q214          1         19       0.95          1      0.995
                  q229          1          2          0          0          0
                  q262          1          1        0.5          1      0.995
                  q265          1          2      0.667          1      0.995
                  q268          2          3          1      0.333      0.667
                  q280          3         18          1          1      0.995
                  q286          2          4        0.8          1      0.995
                  q289          1          8       0.75      0.375      0.516
                  q293          2         16      0.917      0.688       0.83
                  q299          1          2          0          0          0
```

```
                    q31          1         12      0.769      0.833      0.789
                    q61          2          4          1          1      0.995
                     q7          1          2        0.4          1      0.995
                    q73          1          1          1          1      0.995
                    q91          1          4       0.75       0.75      0.656
Speed: 2.6ms preprocess, 12.0ms inference, 0.0ms loss, 1.0ms postprocess per
Results saved to /content/runs/detect/val
```

```
metrics_base_20 = model_base.val(
    data = yaml_path,
    split="test",
    conf=0.20,
    iou=0.6
)
```

```
Ultralytics 8.4.19 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100-S
val: Fast image access ✅ (ping: 0.3±0.1 ms, read: 48.0±15.5 MB/s, size: 80.1
val: Scanning /content/drive/MyDrive/dataset/test/labels.cache... 35 images,
                 Class     Images  Instances      Box(P          R      mAP50
                   all         35        145      0.744      0.805      0.822
                  q106          1          3          1          1      0.995
                  q109          1          3          1          1      0.995
                  q112          2          3          1      0.333      0.667
                  q115          1          1          1          1      0.995
                  q121          1          4      0.444          1      0.912
                   q13          1          8      0.727          1      0.995
                  q142          1          3          1          1      0.995
                   q16          1          1          0          0          0
                  q163          1          1          1          1      0.995
                  q184          1          1          1          1      0.995
                  q187          2          3          1      0.667      0.833
                  q193          1         10      0.529        0.9      0.894
                  q199          1          3          1          1      0.995
                  q202          1          3       0.75          1      0.995
                  q214          1         19       0.95          1      0.995
                  q229          1          2          0          0          0
                  q262          1          1      0.333          1      0.995
                  q265          1          2      0.667          1      0.995
                  q268          2          3          1      0.667      0.833
                  q280          3         18          1          1      0.995
                  q286          2          4        0.8          1      0.995
                  q289          1          8      0.667        0.5      0.531
                  q293          2         16      0.846      0.688      0.819
                  q299          1          2          0          0          0
                   q31          1         12      0.714      0.833      0.784
                   q61          2          4          1          1      0.995
                    q7          1          2        0.4          1      0.995
                   q73          1          1          1          1      0.995
                   q91          1          4       0.75       0.75      0.656
Speed: 4.5ms preprocess, 3.1ms inference, 0.0ms loss, 1.0ms postprocess per i
Results saved to /content/runs/detect/val2
```

```
metrics_base_15 = model_base.val(
    data = yaml_path,
    split="test",
```

```
    conf=0.15,
    iou=0.6
)
```

```
Ultralytics 8.4.19 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100-S
val: Fast image access ✅ (ping: 0.3±0.1 ms, read: 58.6±10.9 MB/s, size: 76.5
val: Scanning /content/drive/MyDrive/dataset/test/labels.cache... 35 images,
                 Class     Images  Instances      Box(P          R      mAP50
                   all         35        145      0.738      0.805      0.822
                 q106          1          3          1          1      0.995
                 q109          1          3          1          1      0.995
                 q112          2          3          1      0.333      0.667
                 q115          1          1          1          1      0.995
                 q121          1          4        0.4          1      0.912
                  q13          1          8      0.727          1      0.995
                 q142          1          3          1          1      0.995
                  q16          1          1          0          0          0
                 q163          1          1          1          1      0.995
                 q184          1          1          1          1      0.995
                 q187          2          3          1      0.667      0.833
                 q193          1         10      0.529        0.9      0.894
                 q199          1          3          1          1      0.995
                 q202          1          3       0.75          1      0.995
                 q214          1         19       0.95          1      0.995
                 q229          1          2          0          0          0
                 q262          1          1      0.333          1      0.995
                 q265          1          2      0.667          1      0.995
                 q268          2          3          1      0.667      0.833
                 q280          3         18          1          1      0.995
                 q286          2          4        0.8          1      0.995
                 q289          1          8      0.667        0.5      0.531
                 q293          2         16      0.846      0.688      0.819
                 q299          1          2          0          0          0
                  q31          1         12      0.714      0.833      0.784
                  q61          2          4          1          1      0.995
                   q7          1          2        0.4          1      0.995
                  q73          1          1          1          1      0.995
                  q91          1          4      0.615       0.75      0.637
Speed: 3.6ms preprocess, 3.1ms inference, 0.0ms loss, 9.0ms postprocess per i
Results saved to /content/runs/detect/val3
```

```
metrics_high_res_25 = model_high_res.val(
    data = yaml_path,
    split="test",
    conf=0.25,
    iou=0.6,
    imgsz = 1024
)
```

```
Ultralytics 8.4.19 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100-S
Model summary (fused): 93 layers, 25,883,764 parameters, 0 gradients, 78.9 GF
val: Fast image access ✅ (ping: 0.3±0.0 ms, read: 63.5±12.8 MB/s, size: 92.4
val: Scanning /content/drive/MyDrive/dataset/test/labels.cache... 35 images,
                 Class     Images  Instances      Box(P          R      mAP50
                   all         35        145      0.729        0.8      0.814
                 q106          1          3       0.75          1      0.995
```

```
                    q109          1          3          1          1      0.995
                    q112          2          3          1      0.667      0.833
                    q115          1          1          1          1      0.995
                    q121          1          4          1          1      0.995
                     q13          1          8      0.727          1      0.995
                    q142          1          3          1          1      0.995
                     q16          1          1          0          0          0
                    q163          1          1          1          1      0.995
                    q184          1          1        0.5          1      0.995
                    q187          2          3      0.333      0.333      0.446
                    q193          1         10        0.6        0.9      0.655
                    q199          1          3          1          1      0.995
                    q202          1          3          1          1      0.995
                    q214          1         19          1      0.947      0.974
                    q229          1          2          0          0          0
                    q262          1          1        0.2          1      0.995
                    q265          1          2      0.667          1      0.995
                    q268          2          3          1      0.667      0.833
                    q280          3         18          1          1      0.995
                    q286          2          4          1          1      0.995
                    q289          1          8      0.556      0.625      0.593
                    q293          2         16      0.714      0.312      0.559
                    q299          1          2          1        0.5       0.75
                     q31          1         12       0.75        0.5      0.579
                     q61          2          4      0.667          1      0.995
                      q7          1          2        0.4          1      0.995
                     q73          1          1          1          1      0.995
                     q91          1          4      0.273       0.75      0.482
Speed: 10.5ms preprocess, 11.5ms inference, 0.0ms loss, 1.5ms postprocess per
Results saved to /content/runs/detect/val4
```

```
metrics_high_res_20 = model_high_res.val(
    data = yaml_path,
    split="test",
    conf=0.20,
    iou=0.6,
    imgsz = 1024
)
```

```
Ultralytics 8.4.19 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100-S
val: Fast image access ✅ (ping: 0.2±0.1 ms, read: 47.4±8.0 MB/s, size: 89.4
val: Scanning /content/drive/MyDrive/dataset/test/labels.cache... 35 images,
                 Class     Images  Instances      Box(P          R      mAP50
                   all         35        145      0.711      0.798      0.814
                  q106          1          3       0.75          1      0.995
                  q109          1          3          1          1      0.995
                  q112          2          3          1      0.667      0.833
                  q115          1          1          1          1      0.995
                  q121          1          4          1          1      0.995
                   q13          1          8      0.728          1      0.995
                  q142          1          3          1          1      0.995
                   q16          1          1          0          0          0
                  q163          1          1          1          1      0.995
                  q184          1          1        0.5          1      0.995
                  q187          2          3      0.333      0.333      0.446
                  q193          1         10      0.598      0.891      0.653
                  q199          1          3          1          1      0.995
                  q202          1          3          1      0.965      0.995
```

```
                    q214          1         19          1      0.944      0.974
                    q229          1          2          0          0          0
                    q262          1          1      0.197          1      0.995
                    q265          1          2      0.667          1      0.995
                    q268          2          3          1      0.667      0.833
                    q280          3         18          1          1      0.995
                    q286          2          4          1          1      0.995
                    q289          1          8      0.579      0.625      0.583
                    q293          2         16      0.714      0.312      0.559
                    q299          1          2          1        0.5       0.75
                     q31          1         12       0.75        0.5      0.579
                     q61          2          4      0.634          1      0.995
                      q7          1          2      0.372          1      0.995
                     q73          1          1      0.547          1      0.995
                     q91          1          4      0.259       0.75      0.475
Speed: 10.1ms preprocess, 6.5ms inference, 0.0ms loss, 1.3ms postprocess per
Results saved to /content/runs/detect/val5
```

```
metrics_high_res_15 = model_high_res.val(
    data = yaml_path,
    split="test",
    conf=0.15,
    iou=0.6,
    imgsz = 1024
)
```

```
Ultralytics 8.4.19 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (NVIDIA A100-S
val: Fast image access ✅ (ping: 0.2±0.1 ms, read: 80.4±15.9 MB/s, size: 101
val: Scanning /content/drive/MyDrive/dataset/test/labels.cache... 35 images,
                 Class     Images  Instances      Box(P          R      mAP50
                   all         35        145      0.705      0.805      0.821
                  q106          1          3       0.75          1      0.995
                  q109          1          3          1          1      0.995
                  q112          2          3          1      0.667      0.833
                  q115          1          1          1          1      0.995
                  q121          1          4      0.837          1      0.995
                   q13          1          8      0.728          1      0.995
                  q142          1          3          1          1      0.995
                   q16          1          1          0          0          0
                  q163          1          1          1          1      0.995
                  q184          1          1        0.5          1      0.995
                  q187          2          3      0.333      0.333      0.446
                  q193          1         10      0.598      0.891      0.653
                  q199          1          3          1          1      0.995
                  q202          1          3          1      0.965      0.995
                  q214          1         19          1      0.944      0.969
                  q229          1          2          0          0          0
                  q262          1          1      0.197          1      0.995
                  q265          1          2      0.667          1      0.995
                  q268          2          3          1      0.667      0.833
                  q280          3         18          1          1      0.995
                  q286          2          4          1          1      0.995
                  q289          1          8      0.579      0.625      0.567
                  q293          2         16      0.714      0.312      0.559
                  q299          1          2          1      0.678      0.995
                   q31          1         12       0.72        0.5      0.577
                   q61          2          4      0.634          1      0.995
```

```
                      q7          1          2      0.372          1      0.995
                     q73          1          1      0.547          1      0.995
                     q91          1          4      0.259       0.75      0.471
Speed: 10.3ms preprocess, 6.2ms inference, 0.0ms loss, 4.2ms postprocess per
Results saved to /content/runs/detect/val6
```

```
def print_metrics(name, metrics):
    print(f"{name:<18} "
          f"{metrics.results_dict['metrics/recall(B)']:.4f}      "
          f"{metrics.results_dict['metrics/precision(B)']:.4f}      "
          f"{metrics.box.map50:.4f}      "
          f"{metrics.box.map:.4f}")

print("\n================ MODEL COMPARISON ================")
print("Model/Conf         Recall     Precision   mAP@0.5    mAP@0.5:0.95")
print("-----------------------------------------------------------------")

# Base Model
print_metrics("Base (0.25)", metrics_base_25)
print_metrics("Base (0.20)", metrics_base_20)
print_metrics("Base (0.15)", metrics_base_15)

print("-----------------------------------------------------------------")

# High-Res Model
print_metrics("HighRes (0.25)", metrics_high_res_25)
print_metrics("HighRes (0.20)", metrics_high_res_20)
print_metrics("HighRes (0.15)", metrics_high_res_15)

print("=================================================================")
```

```
================ MODEL COMPARISON ================
Model/Conf         Recall     Precision   mAP@0.5    mAP@0.5:0.95
-----------------------------------------------------------------
Base (0.25)        0.7774      0.7570      0.8108      0.5311
Base (0.20)        0.8047      0.7441      0.8222      0.5427
Base (0.15)        0.8047      0.7379      0.8216      0.5423
-----------------------------------------------------------------
HighRes (0.25)     0.8001      0.7288      0.8144      0.5522
HighRes (0.20)     0.7985      0.7113      0.8137      0.5517
HighRes (0.15)     0.8046      0.7047      0.8213      0.5509
=================================================================
```

Results Discussion

All the models achieve a recall value more than 67.6, with recall mostly increasing with decrease in confidence thresholds. The high-res model achieves high recall values but also produces lower precision at iou 0.6 when compared to base model. However, mean average precision is higher for HighRes models, indicating better generalisation capability of the models.

Verdict

From the results, the base model with confidence threshold 0.15 and high-res model with confidence threshold of 0.25 are suitable models for the retail stock detection, due to high recall and acceptable precision values. Due to the assignment clearly indicating a high recall and focuses on less precision damaging model, **Base model with 0.15 conf** would be the final chosen model.

```
LO("/content/drive/MyDrive/Retail Assessment/Baseline training/weights/bes
```

Share of Shelf Analytics:

Problem Statement

Treat the entire test dataset as a single representative shelf of a store. Calculate and visualize the "Percentage Share of Shelf" for each of the product classes (SKUs).

Approach

In the next steps, We will map the numerical class IDs (SKU IDs) to their corresponding human-readable class names using the 'names' field from the loaded data.yaml. Then, we will calculate the percentage share for each class by dividing its count by the total number of detected items. Finally, we will visualize the 'Percentage Share of Shelf' for each product class using a bar chart and summarize the analysis, highlighting any dominant product classes.

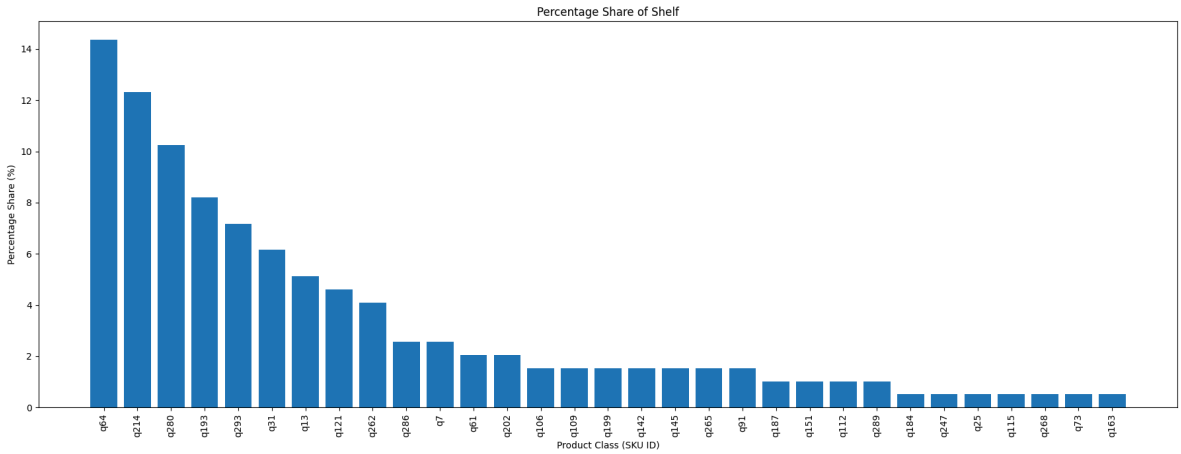
```
from ultralytics import YOLO
from collections import Counter
import os

# Construct the correct path to the test images
test_image_path = os.path.join(data["path"], data["test"])

# model = YOLO("/content/drive/MyDrive/Retail Assessment/Baseline training
results = final_model(test_image_path)

all_labels = []

for r in results:
    labels = r.boxes.cls.cpu().numpy()
    all_labels.extend(labels)

counts = Counter(all_labels)
total = sum(counts.values())
```

```
total = sum(counts.values())

share_of_shelf_data = {}
class_names = data["names"]

for class_id, count in counts.items():
    class_name = class_names[int(class_id)]
    percentage = (count / total) * 100
    share_of_shelf_data[class_name] = percentage
```

```
image 1/35 /content/drive/MyDrive/dataset/test/images/db1050_jpg.rf.6c337978e
image 2/35 /content/drive/MyDrive/dataset/test/images/db1127_jpg.rf.61a026616
image 3/35 /content/drive/MyDrive/dataset/test/images/db1170_jpg.rf.c7c3dd2d1
image 4/35 /content/drive/MyDrive/dataset/test/images/db1205_jpg.rf.c02173b52
image 5/35 /content/drive/MyDrive/dataset/test/images/db1210_jpg.rf.78c93d96b
image 6/35 /content/drive/MyDrive/dataset/test/images/db1212_jpg.rf.6c1c5f6b8
image 7/35 /content/drive/MyDrive/dataset/test/images/db1268_jpg.rf.90d1222a9
image 8/35 /content/drive/MyDrive/dataset/test/images/db1307_jpg.rf.5aef1e6ad
image 9/35 /content/drive/MyDrive/dataset/test/images/db1357_jpg.rf.c179a2bb0
image 10/35 /content/drive/MyDrive/dataset/test/images/db1365_jpg.rf.5cefca58
image 11/35 /content/drive/MyDrive/dataset/test/images/db1379_jpg.rf.f2843907
image 12/35 /content/drive/MyDrive/dataset/test/images/db1415_jpg.rf.56d41732
image 13/35 /content/drive/MyDrive/dataset/test/images/db1440_jpg.rf.e084ae3f
image 14/35 /content/drive/MyDrive/dataset/test/images/db1462_jpg.rf.cf7eeb37
image 15/35 /content/drive/MyDrive/dataset/test/images/db1591_jpg.rf.d9e3e736
image 16/35 /content/drive/MyDrive/dataset/test/images/db1592_jpg.rf.167bb268
image 17/35 /content/drive/MyDrive/dataset/test/images/db1648_jpg.rf.d43fb3d3
image 18/35 /content/drive/MyDrive/dataset/test/images/db1652_jpg.rf.3c7a7f5e
image 19/35 /content/drive/MyDrive/dataset/test/images/db1656_jpg.rf.f7afcf9d
image 20/35 /content/drive/MyDrive/dataset/test/images/db1661_jpg.rf.dcd36dce
image 21/35 /content/drive/MyDrive/dataset/test/images/db1747_jpg.rf.ce58fc13
image 22/35 /content/drive/MyDrive/dataset/test/images/db1758_jpg.rf.9427000f
image 23/35 /content/drive/MyDrive/dataset/test/images/db1761_jpg.rf.7d1fdde9
image 24/35 /content/drive/MyDrive/dataset/test/images/db1763_jpg.rf.986882c7
image 25/35 /content/drive/MyDrive/dataset/test/images/db231_jpg.rf.222a232e2
image 26/35 /content/drive/MyDrive/dataset/test/images/db2760_jpg.rf.f8091045
image 27/35 /content/drive/MyDrive/dataset/test/images/db2783_jpg.rf.3a8a2c89
image 28/35 /content/drive/MyDrive/dataset/test/images/db2837_jpg.rf.d84d18ad
image 29/35 /content/drive/MyDrive/dataset/test/images/db2936_jpg.rf.86e63858
image 30/35 /content/drive/MyDrive/dataset/test/images/db3051_jpg.rf.81f0126c
image 31/35 /content/drive/MyDrive/dataset/test/images/db3081_jpg.rf.497b1cf4
image 32/35 /content/drive/MyDrive/dataset/test/images/db856_jpg.rf.7d91dd44e
image 33/35 /content/drive/MyDrive/dataset/test/images/db859_jpg.rf.d88a1f8ed
image 34/35 /content/drive/MyDrive/dataset/test/images/db871_jpg.rf.1283cd510
image 35/35 /content/drive/MyDrive/dataset/test/images/db956_jpg.rf.fa981329a
Speed: 1.7ms preprocess, 9.5ms inference, 1.3ms postprocess per image at shap
```

```
# Sort by percentage (descending)
sorted_items = sorted(share_of_shelf_data.items(), key=lambda x: x[1], rev

class_labels = [item[0] for item in sorted_items]
percentages = [item[1] for item in sorted_items]

plt.figure(figsize=(18, 7))
plt.bar(class_labels, percentages)
```

```
plt.title('Percentage Share of Shelf')
plt.xlabel('Product Class (SKU ID)')
plt.ylabel('Percentage Share (%)')
plt.xticks(rotation=90)
plt.tight_layout()

plt.savefig("share_of_shelf.png", dpi=300, bbox_inches='tight')

plt.show()
```



Summary of Share of Shelf Analysis

The bar chart above illustrates the 'Percentage Share of Shelf' for each product class (SKU ID) in the test dataset. By examining the chart, we can identify product classes that are dominant on the shelf. These insights are crucial for inventory management, promotional strategies, and understanding product placement effectiveness in a retail environment.

From the visualization, it is clear that **q64** and **q214** are the most dominant product classes, occupying the largest percentages of the shelf. Several other classes like **q280**, **q193**, and **q293** also hold significant shares. Conversely, many classes like **q184**, **q247**, **q25**, **q115**, **q268**, **q73**, and **q163** have a very small percentage, indicating they are less dominant or present in lower quantities. This information can help retailers make informed decisions about stocking levels and merchandising.